Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay

Performance Report (Week 14) in IT 112: Web Systems and Technologies
2nd Semester A.Y 2025 - 2026

NEXUSWrites
"Your Tech, Your Story."

Group Name:
Powerpuffgals

Group Members:
Kris Ann B. Apoon
April Grace R. Dacanay
Ivy Angel R. Hidalgo
Princes Mae L. Mateo
Milan R. Rellora

Course & Block:
BSIT-2A

Introduction

In this phase of NEXUSWrites, the website becomes more engaging as it starts displaying real blog content retrieved from the backend. Instead of static text, blog posts and related data are now loaded dynamically and presented on the webpage. This allows users to view actual content, making the platform feel more active and closer to a real blogging site.

This stage is important because it improves both the functionality and experience of the website. By using JavaScript and DOM manipulation, blog posts can be displayed in an organized and visually appealing way. Features such as structured layouts and optional search tools help users easily navigate through different posts, making the platform more interactive and user-friendly.

Objectives

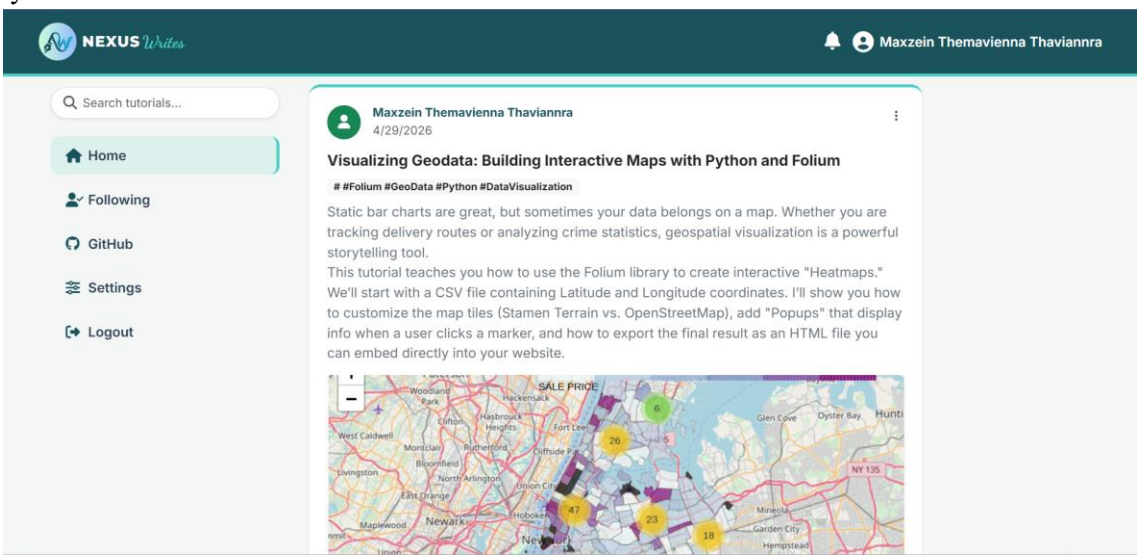
By the end of this phase, we were able to:

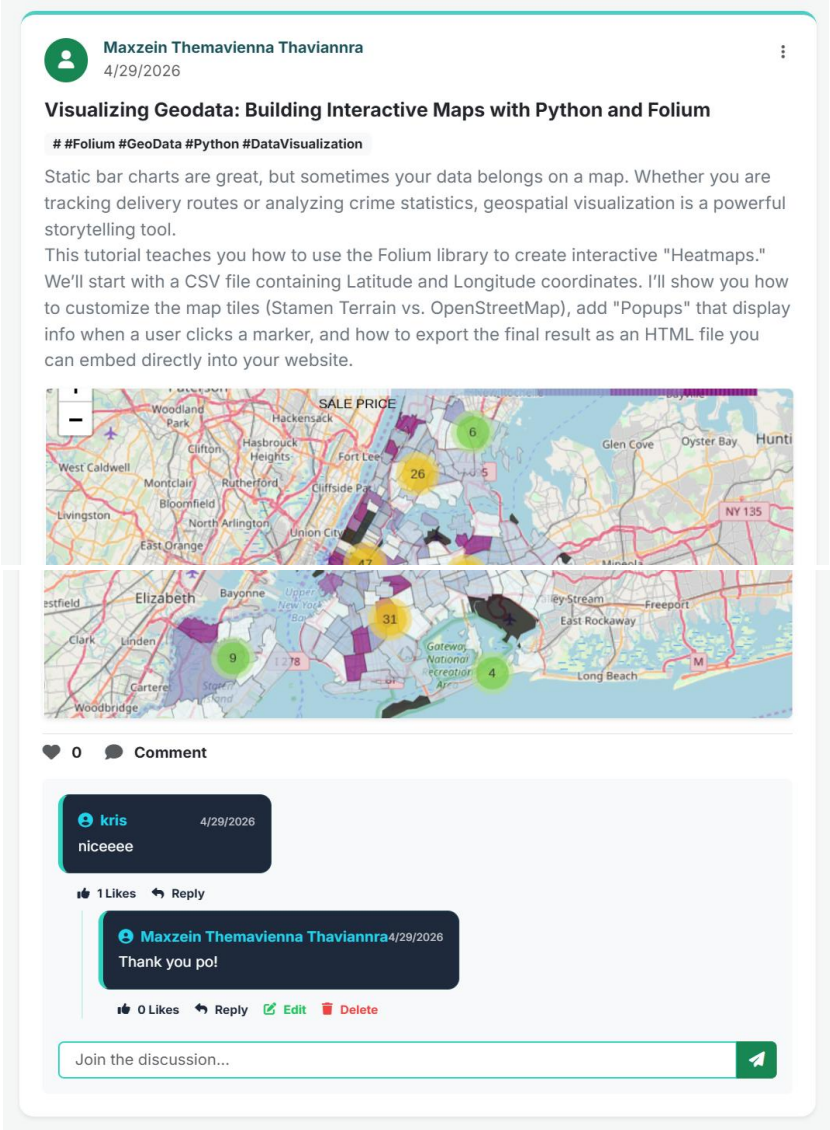
- Retrieve blog data (such as posts and user information) from backend APIs using fetch() or axios.
- Dynamically display blog content on the webpage using DOM manipulation (e.g., blog cards or post lists).
- Design and apply reusable templates for a consistent and organized blog interface.
- Implement basic search or content filtering to improve navigation and user experience.
- Manage and organize frontend and backend updates with clear and proper GitHub commits.

I. Display Pages

- **Dashboard and posts**

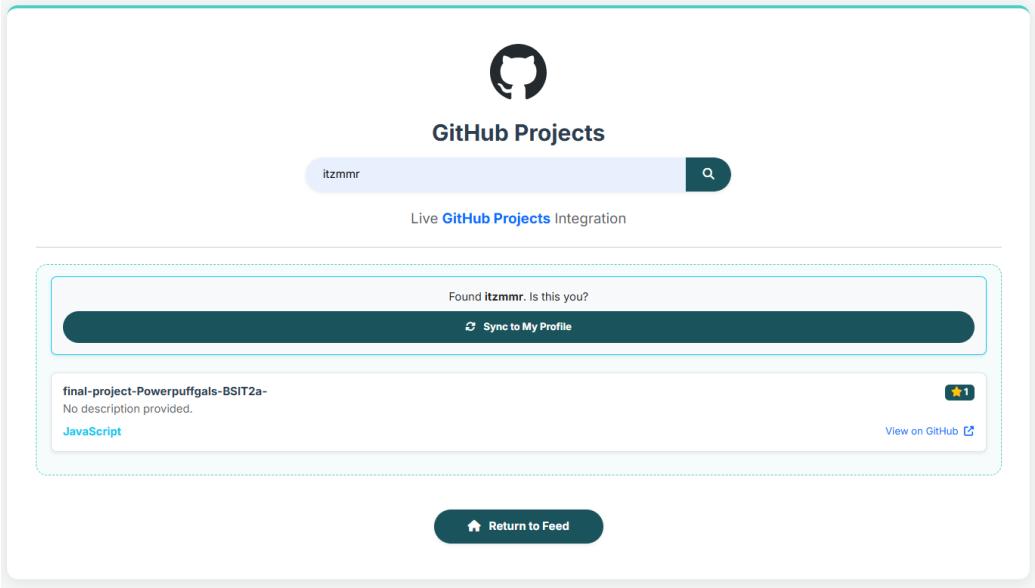
The NexusWrites dashboard displays posts with complete details such as author information, title, tags, content, and embedded images, creating an organized and informative feed. It also supports user interaction through likes, comments, replies, and editable comment sections, making the platform engaging and dynamic.

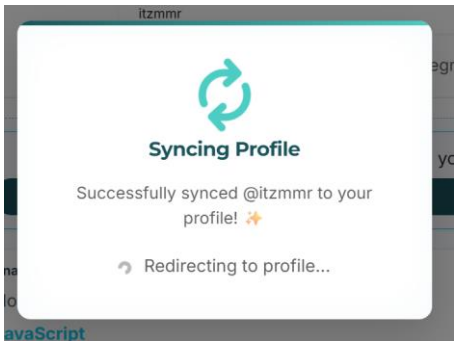


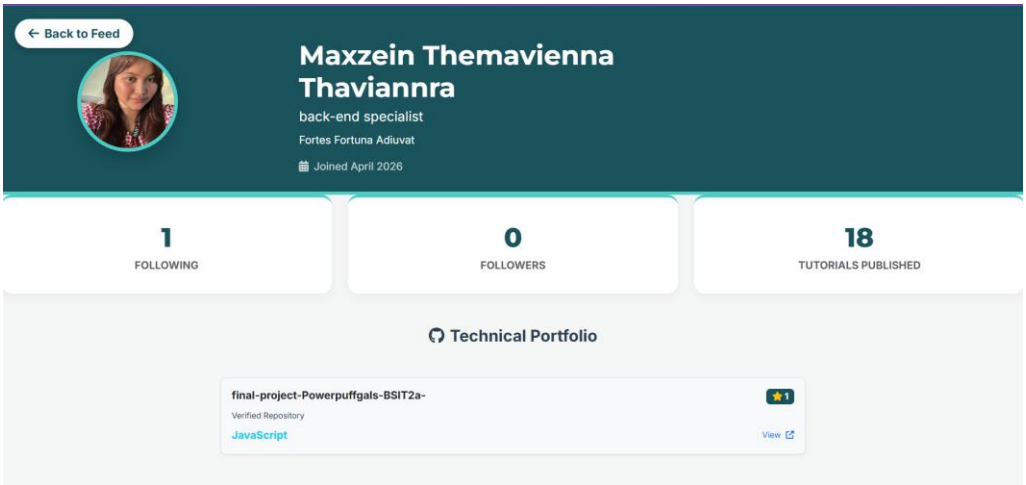


- **GitHub Integration**

The GitHub integration feature allows users to search their repositories and select projects to connect with their NexusWrites profile. Once synced, these repositories are displayed in the user's profile under the technical portfolio section, showcasing their work.


GitHub Projects
itzmmr
Live GitHub Projects Integration
Found itzmmr. Is this you?
Sync to My Profile
final-project-Powerpuffgals-BSIT2a-
No description provided.
JavaScript
View on GitHub
Return to Feed

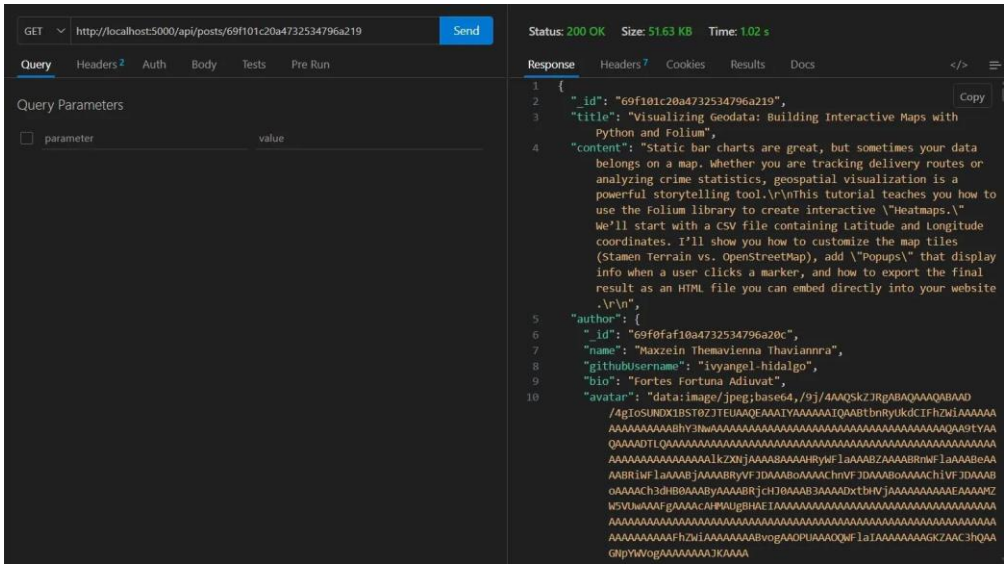

itzmmr
Syncing Profile
Successfully synced @itzmmr to your profile!
Redirecting to profile...


Back to Feed
Maxzein Themavienna Thaviannra
back-end specialist
Fortes Fortuna Adiuvat
Joined April 2026
1
FOLLOWING
0
FOLLOWERS
18
TUTORIALS PUBLISHED
Technical Portfolio
final-project-Powerpuffgals-BSIT2a-
Verified Repository
JavaScript
View

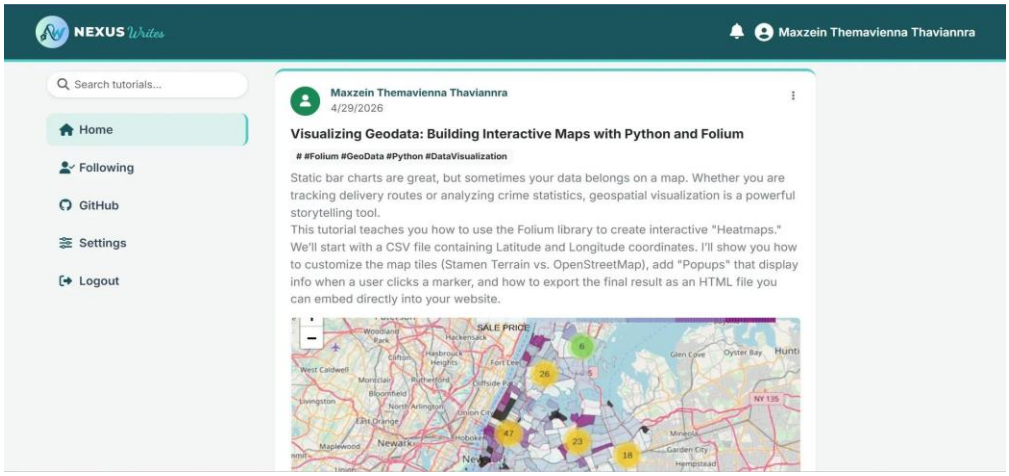
II. Sample backend API response

- **Sample backend API response in thunder (get post by id)**

The "get post by ID" request returns an OK status, meaning the specific post was successfully found and fetched by the server. The response provides a single JSON object with detailed information about that particular post, such as its title, content, and auth**or.**

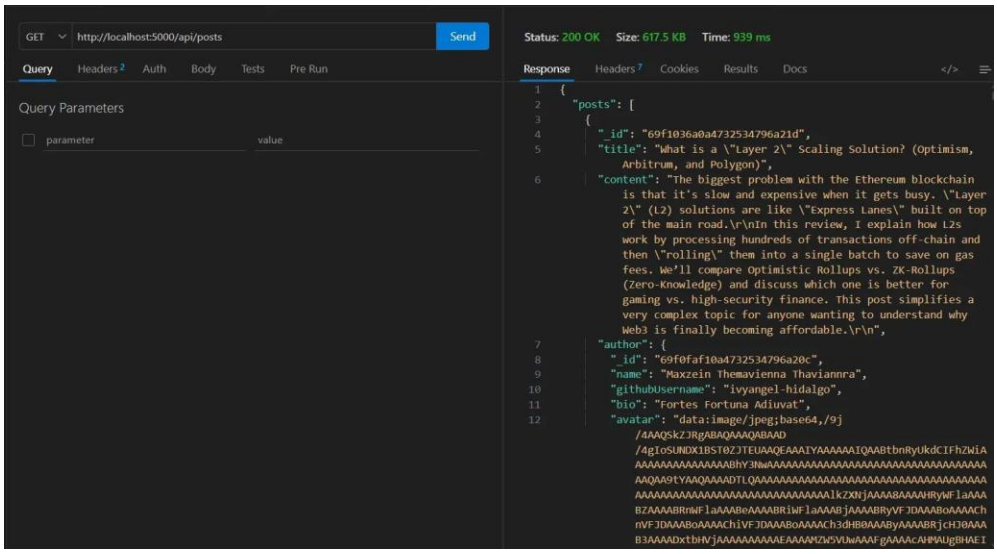


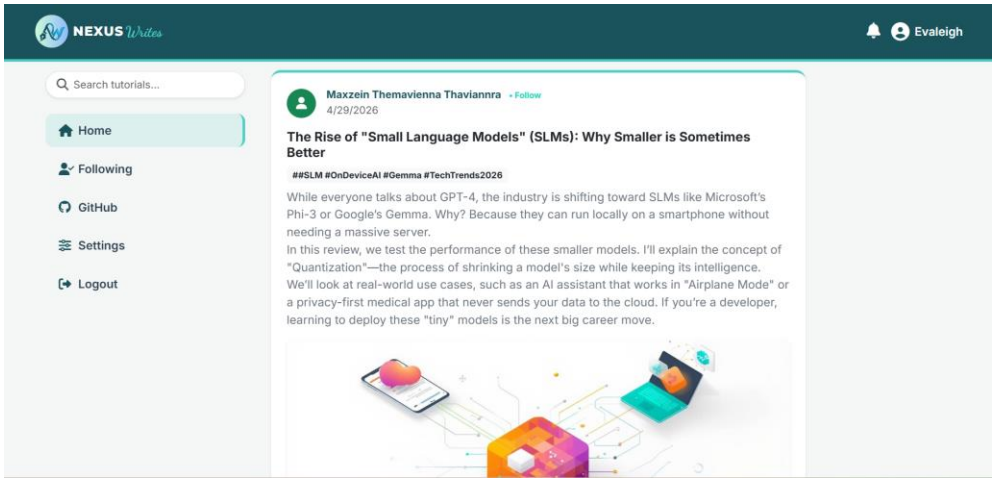
Results:



- **get all post**

The request for "get all posts" shows an OK status, indicating that the API successfully retrieved all available posts from the database. The response body contains a structured JSON array with multiple post objects, each including details like title, content, and author.


GET
http://localhost:5000/api/posts
Send
Query
Headers 2
Auth
Body
Tests
Pre Run
Query Parameters
parameter
value
Status: 200 OK
Size: 617.5 KB
Time: 939 ms
Response
Headers 7
Cookies
Results
Docs
{
"posts": [
{
"_id": "69f1036a0a4732534796a21d",
"title": "What is a \"Layer 2\" Scaling Solution? (Optimism, Arbitrum, and Polygon)",
"content": "The biggest problem with the Ethereum blockchain is that it's slow and expensive when it gets busy. \"Layer 2\" (L2) solutions are like \"Express Lanes\" built on top of the main road.\r\nIn this review, I explain how L2s work by processing hundreds of transactions off-chain and then \"rolling\" them into a single batch to save on gas fees. We'll compare Optimistic Rollups vs. ZK-Rollups (Zero-Knowledge) and discuss which one is better for gaming vs. high-security finance. This post simplifies a very complex topic for anyone wanting to understand why Web3 is finally becoming affordable.\r\n",
"author": {
"_id": "69f0faf10a4732534796a20c",
"name": "Maxzein Themavienna Thaviannra",
"githubUsername": "ivyangel-hidalgo",
"bio": "Fortes Fortuna Adiuvat",
"avatar": "



III. Annotated JavaScript snippet used for rendering data

- **dashboard.js**

The renderSinglePost(post, user) function in dashboard.js builds each post in the feed, including the author info, content, tags, image, likes, and comments. It also uses escapeHtml() for security, shows owner-only controls when needed, and integrates the comment rendering function.

```
function renderSinglePost(post, user) {
    const isOwner = user && (
        (post.author?._id && post.author._id === user._id) ||
        (post.author === user._id)
    );
    const isFollowingUser = post.author?.followers?.includes(user?._id);
    const followLink = (!isOwner && post.author) ? `
        <span class="ms-2 small fw-bold follow-link"
              id="follow-link-${post.author?._id || post.author}"
              style="cursor: pointer; color: #2dd4bf; font-size: 0.8rem;"
              onclick="handleFollowAction('${post.author?._id || post.author}')">
            ${isFollowingUser ? '• Following' : '• Follow'}
        </span>` : '';
    const tagsHTML = (post.tags && Array.isArray(post.tags))
        ? post.tags.map(tag => `<span class="badge bg-light text-dark me-1">#${escapeHtml(tag)}</span>`).join('')
        : (typeof post.tags === 'string' && post.tags.trim() !== '')
            ? post.tags.split(',').map(tag => `<span class="badge bg-light text-dark me-1">#${escapeHtml(tag.trim())}</span>`).join('')
            : '';

    return `
    <div class="card fb-card p-4 mb-4 shadow-sm border-0">
        ${isOwner ? `
        <div class="dropdown" style="position: absolute; right: 20px; top: 20px; z-index: 10;">
            <button class="btn btn-link text-muted" data-bs-toggle="dropdown"><i class="fas fa-ellipsis-v"></i></button>
            <ul class="dropdown-menu dropdown-menu-end shadow border-0">
                <li><a class="dropdown-item" onclick="editPost('${post._id}')"><i class="fas fa-edit me-2 text-success"></i>Edit Post</a></li>
                <li><a class="dropdown-item text-danger" onclick="deletePost('${post._id}')"><i class="fas fa-trash me-2"></i>Delete Post</a></li>
            </ul>
        </div>` : ''}

        <div class="d-flex align-items-center mb-3">
            <div class="rounded-circle bg-success text-white d-flex align-items-center justify-content-center me-3" style="width: 45px; height: 45px;">
                <i class="fas fa-user"></i>
            </div>
            <div>
```

- **github.js**

This script transforms raw JSON data into a visual UI by first clearing the container and adding a "Sync" header with a repository list placeholder. It then iterates through the project array, dynamically injecting Bootstrap-styled cards that display the name, stars, description, language, and links for each repository.

```
            container.innerHTML = `
                <div class="mb-4 p-3 rounded-3 bg-light border border-info text-center shadow-sm">
                    <p class="small mb-2 text-dark">Found <strong>${username}</strong>. Is this you?</p>
                    <button class="btn btn-nexus-primary btn-sm w-100" onclick="syncThisAccount('${username}')">
                        <i class="fas fa-sync-alt me-2"></i> Sync to My Profile
                    </button>
                </div>
                <div class="text-start" id="repo-list"></div>`;
            const list = document.getElementById('repo-list');
            repos.forEach(repo => {
                list.innerHTML += `
                    <div class="repo-card p-3 rounded-3 mb-3 shadow-sm border">
                        <div class="d-flex justify-content-between">
                            <h6 class="fw-bold mb-1 repo-title">${repo.name}</h6>
                            <span class="badge" style="background-color: var(--nexus-teal)">
                                <i class="fas fa-star text-warning"></i> ${repo.stars}
                            </span>
                        </div>
                        <p class="small text-muted mb-2">${repo.description || 'No description provided.'}</p>
                        <div class="d-flex justify-content-between align-items-center">
                            <small class="fw-bold text-info">${repo.language || 'Plain Text'}</small>
                            <a href="${repo.url}" target="_blank" class="btn btn-sm btn-link text-decoration-none p-0">
                                View on GitHub <i class="fas fa-external-link-alt ms-1"></i>
                            </a>
                        </div>
                    </div>`;
            });
        } catch (err) {
            container.innerHTML = `<p class="text-danger fw-bold">Error: ${err.message}. Please check your backend connection.</p>`;
        }
    }
```

- **profile.js**

Each post becomes a distinct card with a green left border, title, category, content, optional image, and like/comment counters. This is how the profile page displays the user's own tutorials.

```
container.innerHTML = posts.map(post => `
    <div class="row">
        <div class="col-12">
            <div class="post-card mb-5 shadow-sm p-4" style="background: #fff; border-radius: 20px; border-left: 8px solid #1a535c;">
                <div class="d-flex justify-content-between mb-3">
                    <h5 class="fw-bold" style="color: #1a535c;">${post.title}</h5>
                    <span class="badge" style="background: #5b6d5b;">${post.category || 'Web Development'}</span>
                </div>

                <p class="text-secondary mb-3">${post.content}</p>

                ${post.image ? `
                    <div class="bg-light p-2 rounded-3 mb-3">
                        <img src="${post.image}" class="img-fluid rounded"
                             style="width: 100%; object-fit: cover; max-height: 450px;">
                    </div>`
                : ''}

                <div class="d-flex gap-3 pt-3 border-top">
                    <small class="text-muted"><i class="fas fa-heart text-danger"></i> ${post.likes?.length || 0}</small>
                    <small class="text-muted"><i class="fas fa-comment text-success"></i> ${post.comments?.length || 0}</small>
                </div>
            </div>
        </div>
    </div>
`).join('');
```

A responsive grid of GitHub repository cards appears on the profile page, each showing the repo name, star count, description, language, and a link to view the code.

```
display.innerHTML = `<div class="row g-3 m-0 p-0 justify-content-center" id="repo-grid"></div>`;
const grid = document.getElementById('repo-grid');

repos.forEach(repo => {
    grid.innerHTML += `
        <div class="col-12 p-0 mb-3">
            <div class="p-3 border rounded-3 bg-body-tertiary repo-card-custom shadow-sm h-100 position-relative">
                <div class="d-flex justify-content-between align-items-start text-start">
                    <h6 class="fw-bold text-truncate mb-0 text-body" style="max-width: 80%;">${repo.name}</h6>
                    <span class="badge" style="background-color: #1a535c;">
                        <i class="fas fa-star text-warning"></i> ${repo.stars || 0}
                    </span>
                </div>

                <p class="small text-secondary-custom my-2 text-start" style="font-size: 0.75rem;">
                    ${repo.description || 'Verified Repository'}
                </p>

                <div class="d-flex justify-content-between align-items-center mt-2">
                    <span class="small fw-bold text-info">${repo.language || 'Code'}</span>
                    <a href="${repo.url}" target="_blank" class="btn btn-sm btn-link p-0 text-decoration-none" style="font-size: 0.75rem;">
                        View <i class="fas fa-external-link-alt ms-1"></i>
                    </a>
                </div>
            </div>
        </div>`;
});
```

- **app.js**

This function maps a commentList and postId into a threaded UI by rendering the author, metadata, and action buttons for each entry. It uses recursion to process comment.replies, ensuring nested conversations are displayed in a structured, hierarchical format.

```
function renderComments(commentList, postId) {
    if (!commentList || commentList.length === 0) return '';

    return commentList.map((comment) => {
        const cId = comment._id.toString();
        const pId = postId.toString();

        const user = JSON.parse(localStorage.getItem('nexusUser'));
        const isCommentOwner = user && (
            (comment.user?._id && comment.user._id === user._id) ||
            (comment.user === user._id)
        );

        return `
        <div class="comment-thread mb-3" style="position: relative;">
            <div class="comment-box shadow-sm p-3 rounded bg-light border-start border-success border-4">
                <div class="d-flex justify-content-between align-items-center">
                    <strong class="text-success">${comment.user?.name || 'Nexus Writer'}</strong>
                    <small class="text-muted">${new Date(comment.createdAt).toLocaleDateString()}</small>
                </div>
                <p class="mb-0 mt-1" id="text-${cId}" style="font-size: 0.95rem;">${comment.text}</p>
            </div>

            <div class="comment-actions mt-1 ms-2 d-flex gap-3" style="font-size: 0.85rem; position: relative; z-index: 10;">
                <span class="action-btn text-muted" onclick="toggleCommentLike('${pId}', '${cId}')" style="cursor: pointer;">
                    <i class="fas fa-thumbs-up"></i> ${comment.likes?.length || 0} Likes
                </span>

                <span class="action-btn text-primary" onclick="toggleReplyInput('${pId}', '${cId}')" style="cursor: pointer;">
                    <i class="fas fa-reply"></i> Reply
                </span>
```

This logic processes notification objects containing metadata like type, sender, and read status into distinct visual cards featuring avatars and type-specific icons. It generates a complete feed by mapping the

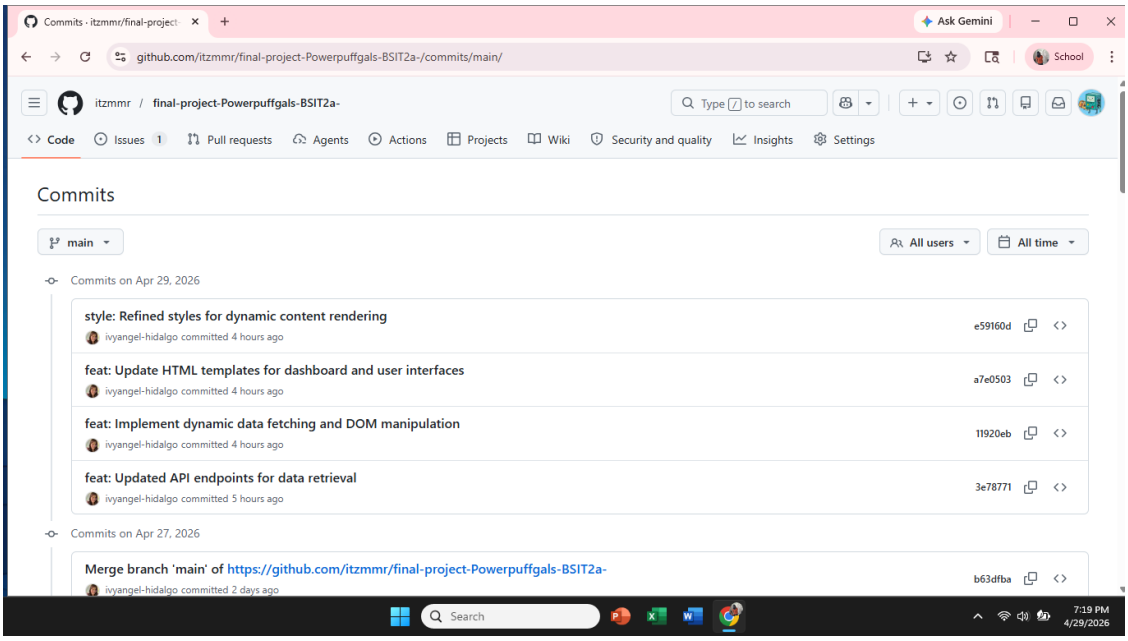
renderNotificationItem function across the data array to produce a timestamped, descriptive list of user interactions.

```
function renderNotificationItem(n) {
    const unreadClass = n.isRead ? '' : 'notification-unread';
    const senderName = n.sender ? n.sender.name : 'Someone';
    const senderAvatar = (n.sender && n.sender.avatar) ? n.sender.avatar : 'https://via.placeholder.com/45';
    const timeAgo = new Date(n.createdAt).toLocaleString();

    let actionText = "";
    let icon = "";

    switch(n.type) {
        case 'like':
            actionText = `liked your post <span class="fw-bold text-dark">"${n.post?.title || 'Untitled Post'}"</span>`;
            icon = '<i class="fas fa-heart text-danger small"></i>';
            break;
        case 'comment':
            actionText = `commented on your post <span class="fw-bold text-dark">"${n.post?.title || 'Untitled Post'}"</span>`;
            icon = '<i class="fas fa-comment text-primary small"></i>';
            break;
        case 'follow':
            actionText = `started following you`;
            icon = '<i class="fas fa-user-plus text-success small"></i>';
            break;
        default:
            actionText = `interacted with your profile`;
            icon = '<i class="fas fa-info-circle text-secondary small"></i>';
    }

    return `
        <div class="list-group-item list-group-item-action p-3 notification-card ${unreadClass} border-bottom">
            <div class="d-flex align-items-center">
                <div class="position-relative">
                    <img src="${senderAvatar}" class="rounded-circle avatar-img me-3 shadow-sm" alt="User">
                    <span class="position-absolute bottom-0 end-0 bg-white rounded-circle p-1" style="transform: translate(-10px, 5px)
                        ${icon}
                    </span>
```

This function reads a selected file as a Base64 Data URL to provide an instant visual preview by updating an image element's source. It refines the UI by hiding placeholder icons and capturing the encoded string, ensuring the user's input is immediately rendered and prepared for backend submission.

```
let base64Image = "";

function previewImage(input) {
    if (input.files && input.files[0]) {
        const reader = new FileReader();
        reader.onload = (e) => {
            const preview = document.getElementById('avatarPreview');
            preview.src = e.target.result;
            preview.style.display = 'block';
            document.getElementById('avatarIcon').style.display = 'none';
            base64Image = e.target.result;
        };
        reader.readAsDataURL(input.files[0]);
    }
}
```

IV. GitHub Commits Logs



V. Member Reflections

Ivy Angel Hidalgo (Project Manager & Backend Developer)
In this phase, I focused on ensuring that all GET endpoints return accurate and well-structured data that the frontend can easily use. I guided the team in connecting the backend to the user interface and made sure everything worked smoothly during testing. There were a few challenges with data formatting, but I addressed them carefully through testing and adjustments. This experience helped me better appreciate the importance of consistency between the backend and frontend for a more reliable system.

Kris Ann Apoon (Frontend Developer)
During this phase, I worked on building dynamic HTML interfaces using the data received from the backend. I made sure that the information is displayed clearly and updates properly on the page. At first, handling and organizing the data was a bit challenging, but I improved it through continuous testing and practice. This phase helped me become more confident in creating responsive and user-friendly interfaces.

April Grace Dacanay (Documentation Officer)
In this phase, I focused on documenting how the system outputs data and how it is displayed on the frontend. I included screenshots of the actual results and added clear explanations to make the process easier to understand. I made sure the documentation is simple, organized, and informative.

This experience helped me improve my ability to present technical details in a clear and structured way.

Princes Mae Mateo (Database Manager)
During this phase, I ensured that the data retrieved by the GET endpoints is accurate, complete, and properly structured. I carefully reviewed the stored records and made necessary adjustments to maintain consistency. Although there were minor issues, I was able to resolve them step by step. This phase helped me realize how important well-organized data is in supporting the overall performance of the system.

Milan Rellora (GitHub Manager)
In this phase, I managed the merging of updates into the main branch and ensured that all changes were integrated smoothly. I reviewed each update carefully to avoid conflicts and maintain the stability of the project. When conflicts occurred, I handled them patiently and made sure everything was resolved properly. This experience helped me improve my skills in managing collaboration and maintaining an organized workflow.